

Explainable RUL Prediction for Aerospace Li-Ion Batteries

Using Attention-Based LSTM

Mahmut Esat Kolay

77th International Astronautical Congress 2026 | IAC-26 E2.1

Paper ID: 105214

1. Abstract

Lithium-ion batteries are critical components of spacecraft power subsystems and small satellite missions, where unexpected degradation can compromise mission safety and operational lifespan. Accurate prediction of Remaining Useful Life (RUL) is therefore essential for improving reliability and enabling predictive maintenance in aerospace energy systems. Although deep learning models such as Long Short-Term Memory (LSTM) networks have demonstrated strong performance in battery prognostics, they often suffer from limited interpretability and reduced robustness when operational data are scarce or degradation occurs rapidly.

This study proposes a hybrid deep learning framework for RUL prediction using the NASA Ames PCoE battery dataset under varying thermal conditions (4 °C, 24 °C, and 43 °C). The main contributions of this work are threefold: (1) the integration of an Attention-based LSTM architecture that improves temporal feature learning for battery degradation modeling, (2) the development of an explainable AI framework that visualizes the temporal importance of sensor signals, providing interpretable insights into the degradation process, and (3) an Adaptive Input Strategy that enables early-cycle prognostics through dynamic edge-padding, allowing the model to generate predictions even when limited operational data are available.

The proposed approach is evaluated against conventional sequence-based prediction strategies to assess its robustness under diverse degradation scenarios. Experimental results demonstrate strong predictive performance, achieving an overall Root Mean Square Error (RMSE) of 11.92 cycles and an R^2 score of 0.9319 across different thermal stress conditions. In rapid degradation cases where sequence lengths are shorter than standard window sizes, the Adaptive Input Strategy significantly improves prediction robustness, reducing RMSE from 117 cycles to 18.79 cycles (approximately 84% improvement) compared with traditional fixed-window approaches.

The results indicate that the proposed framework provides an accurate and interpretable prognostics solution for aerospace lithium-ion battery systems. By combining deep learning with explainable modeling and adaptive data handling, this work supports the development of more reliable battery health monitoring strategies for future spacecraft and aerospace energy systems.

2. Introduction

Battery health monitoring is a cornerstone of reliable spacecraft operations. In small satellite and CubeSat platforms, the power subsystem is a single point of failure — battery degradation without warning can terminate a mission prematurely. The Remaining Useful Life (RUL) of a battery, defined as the number of charge-discharge cycles remaining until capacity falls below a predefined failure threshold (commonly 70–80% of rated capacity), is the key metric for predictive maintenance scheduling.

Traditional physics-based models require detailed electrochemical parameterisation that is difficult to obtain in-orbit. Data-driven approaches, particularly deep learning methods, have emerged as effective alternatives. Among these, Long Short-Term Memory (LSTM) networks excel at capturing long-range temporal dependencies in cycle-by-cycle degradation signals. However, standard LSTM models present two critical limitations for aerospace deployment: they are opaque (offering no insight into which cycles or features drove a prediction), and they require a full history window before producing any output, making them blind during early mission phases.

This work addresses both limitations through a unified framework: an Attention-LSTM architecture that weights each time step's contribution to the final RUL estimate, combined with an Adaptive Input Strategy (AIS) that uses edge-padding to enable predictions from the very first operational cycle. The attention weights are directly visualized as explainability maps, making the model's temporal reasoning transparent to mission operators.

3. Dataset

3.1 NASA Ames PCoE Battery Dataset

The study uses the publicly available NASA Ames Prognostics Center of Excellence (PCoE) battery dataset. This benchmark dataset contains charge, discharge, and impedance measurements for 18650-format lithium-ion cells cycled to end-of-life under three controlled ambient temperatures: 4 °C (cold), 24 °C (nominal), and 43 °C (elevated). Three representative batteries are selected for evaluation:

- B0005 — 24 °C ambient; 168 discharge cycles; gradual degradation profile
- B0032 — 43 °C ambient; only ~10 discharge cycles; extremely rapid capacity fade
- B0044 — 4 °C ambient; ~82 discharge cycles; moderate cold-stress degradation

Each discharge record contains time-series measurements of voltage, current, and temperature sampled at approximately 10 Hz. The end-of-life criterion is defined as the point at which the discharge capacity drops below 1.4 Ahr (approximately 70% of the nominal 2.0 Ahr capacity). The RUL label for each cycle is computed as the number of cycles remaining until this threshold is crossed.

3.2 Data Loading Pipeline

Raw data is stored in MATLAB .mat files, one per battery cell. The `data_loader.py` module implements `scipy.io.loadmat`-based extraction. The loader navigates the nested MATLAB struct hierarchy, filters only discharge-type cycles, and extracts the capacity scalar along with voltage, current, temperature, and time arrays for each cycle.

```
# data_loader.py - core extraction logic
col = data[battery_id][0][0][0][0]
for i in range(col.shape[0]):
    if operation_type == 'discharge':
        capacity = measurements[6][0][0]
        voltage = measurements[0][0]
        current = measurements[1][0]
        temperature = measurements[2][0]
        time_sec = measurements[5][0]
```

After loading, cycles are re-indexed sequentially (1, 2, 3, ...) to remove gaps caused by interleaved charge and impedance cycles. A `load_all_data()` helper recursively searches subdirectories (24C/, 43C/, 04C/) to batch-process the full dataset.

4. Feature Engineering

4.1 Feature Extraction

Rather than feeding raw time-series arrays into the model (which would require a variable-length inner sequence per cycle), four cycle-level summary statistics are extracted that are physically motivated degradation indicators. These are implemented in the `DataProcessor` class in `feature_eng.py`.

Feature	Source Signal	Physical Meaning	Degradation Indicator
discharge_time	time_sec	Total discharge duration	Decreases as capacity fades
min_voltage	voltage_measured	Minimum recorded voltage	Drops with cell aging
max_temp	temperature_measured	Peak cycle temperature	Rises with internal resistance
ambient_temp	metadata	Operating environment temp	Context for 4 °C/24 °C/43 °C

Table 1 — Extracted features, their raw signal origins, physical meaning, and role as degradation indicators.

Discharge time is calculated as the difference between the last and first timestamps of the `time_sec` vector for each cycle. This scalar decreases monotonically as capacity fades — a shorter discharge indicates the cell can no longer sustain current for as long. Minimum voltage captures the deepest voltage sag during a cycle, which deepens with cell aging due to rising internal impedance. Maximum temperature reflects peak thermal generation, which increases as the cell's internal resistance grows. Ambient temperature provides critical environmental context, as the same intrinsic degradation rate manifests very differently at 4 °C versus 43 °C.

4.2 RUL Label Construction

For each battery, the total number of discharge cycles `N` is determined. The RUL for cycle `i` is then simply:

```
RUL[i] = total_cycles - cycle_count[i]
```

This linear countdown label is standard in the battery prognostics literature and assumes that the end-of-life event is reached at the final recorded discharge cycle. The resulting feature table is saved to `data/processed/final_features.csv` to avoid redundant recomputation on subsequent runs.

4.3 Normalisation

All four features are normalised to `[0, 1]` using `MinMaxScaler` from `scikit-learn`. Normalisation is fit exclusively on training data to prevent information leakage from the test set. The scaler object is retained in the `DataProcessor` instance and reapplied consistently during evaluation.

5. Model Architecture

5.1 Overview

The model follows an Encoder-Attention-Regressor pattern. A two-layer LSTM encodes the temporal dynamics of the 30-cycle input window into a sequence of hidden states. An additive attention mechanism then computes a weighted context vector that emphasises the most degradation-relevant time steps. Finally, a two-layer fully connected head maps this context to a scalar RUL estimate.

5.2 LSTM Encoder

The LSTM layer processes the input tensor of shape (Batch, 30, 4) — 30 cycles, 4 features — and produces hidden states of shape (Batch, 30, 64). Two stacked LSTM layers with dropout=0.2 are used. The stacked structure allows the upper layer to capture higher-order temporal patterns such as the acceleration of degradation that often precedes end-of-life. The final hidden state (h_n) is not used directly; instead, the full hidden state sequence is passed to the attention module.

```
# model.py - LSTM definition
self.lstm = nn.LSTM(
    input_size = 4,
    hidden_size = 64,
    num_layers = 2,
    batch_first = True,
    dropout = 0.2
)
```

5.3 Attention Mechanism

The Attention class implements Bahdanau-style additive attention. For each of the 30 time steps in the hidden state sequence, a scalar energy score is computed via a linear transformation followed by tanh activation. A final linear projection reduces each 64-dimensional energy vector to a scalar, which is then normalised across the window dimension by softmax to produce the attention weight vector α of shape (Batch, 30, 1). The context vector is the attention-weighted sum of the hidden states:

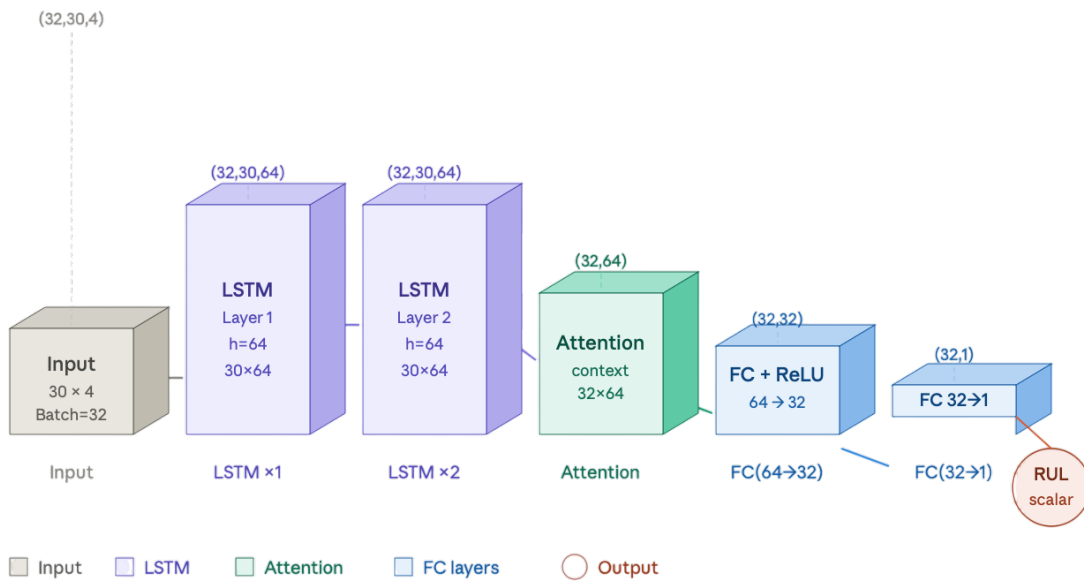
```
# model.py - Attention forward pass
energy = torch.tanh(self.attn(hidden_states)) # (B, 30, 64)
attention = self.v(energy) # (B, 30, 1)
alpha = F.softmax(attention, dim=1) # (B, 30, 1)
context = torch.sum(hidden_states * alpha, dim=1) # (B, 64)
```

The α weights are returned alongside the RUL prediction, enabling direct visualisation of the temporal importance profile — the explainability component of the framework.

5.4 Regression Head

The 64-dimensional context vector is passed through a two-stage fully connected head: $\text{Linear}(64 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{Linear}(32 \rightarrow 1)$. No activation is applied to the final output, allowing the model to produce unbounded scalar RUL estimates appropriate for regression.

5.5 Architecture Summary



The proposed model is a sequence-to-scalar deep learning architecture designed for Remaining Useful Life (RUL) prediction. It accepts multivariate time-series input of shape (Batch=32, Window=30, Features=4), processing 30 time steps across 4 sensor features per sample. The temporal dependencies within the input sequence are captured by two stacked LSTM layers, each with a hidden size of 64 and a dropout rate of 0.2 to mitigate overfitting. Following the recurrent layers, a soft attention mechanism computes a weighted context vector over all 30 time steps, effectively allowing the model to focus on the most informative temporal positions rather than relying solely on the final hidden state. The resulting context vector is then passed through two fully connected layers — FC(64→32) with ReLU activation, followed by FC(32→1) — which project the learned representation down to a single scalar output corresponding to the predicted RUL value.

6. Training

6.1 Data Pipeline

Sliding windows of length 30 are extracted from each battery's normalised feature sequence. For a battery with N cycles, this produces $N - 30$ windows. The target label for each window is the RUL value at the cycle immediately following the window (i.e., a one-step-ahead prediction horizon). All batteries in the training set contribute windows, giving a heterogeneous training corpus that spans all three thermal conditions.

6.2 Hyperparameters

Hyperparameter	Value	Rationale
Batch Size	32	Standard mini-batch; balances speed vs. gradient quality
Epochs	100	Sufficient convergence; best model saved via checkpoint
Learning Rate	0.001	Adam default; stable for LSTM regression tasks
Window Size	30	Captures ~1 month of degradation history
Hidden Dim	64	Compact yet expressive for tabular time-series
LSTM Layers	2	Stacked layers for hierarchical temporal features
Dropout	0.2	Regularisation to prevent overfitting on small dataset
Optimiser	Adam	Adaptive LR; well-suited to sequence models
Loss Function	MSELoss	Penalises large RUL errors; standard for regression
Test Split	20 %	Stratified random split via sklearn

Table 2 — Training hyperparameters and their rationale.

6.3 Training Loop

Training uses a standard mini-batch gradient descent loop. At each epoch, the model processes all training batches, computes MSE loss, backpropagates gradients, and updates weights via Adam. After each epoch, validation loss is computed on the held-out 20% split. The best model checkpoint (lowest validation loss) is saved to `models/best_model.pth`. This ensures the deployed model represents the optimal generalisation point rather than the fully converged but potentially overfit end-epoch state.

```
# train.py - checkpoint logic
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    torch.save(model.state_dict(), '../models/best_model.pth')
```

6.4 Loss Curve Analysis

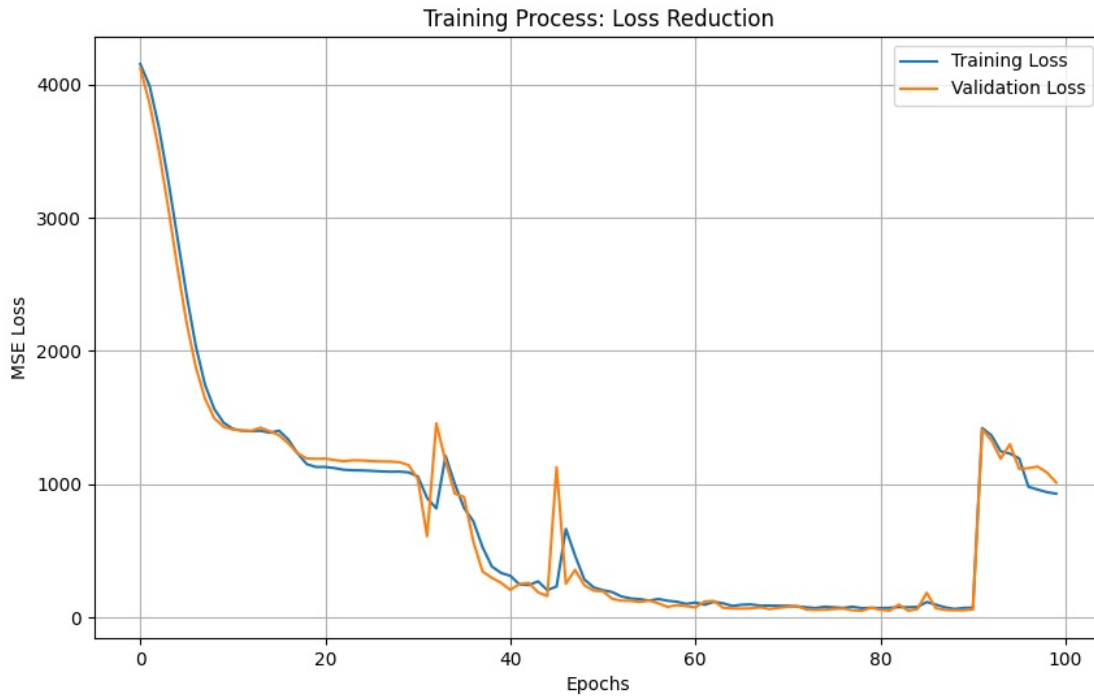


Figure 1 — Training and validation MSE loss over 100 epochs.

The loss curve (Figure 1) reveals several notable features. Both training and validation losses decline steeply from an initial MSE of ~4100 to below 400 within the first 45 epochs, indicating rapid gradient descent in the early learning phase. Short-lived loss spikes are visible at approximately epochs 30–32 and 43–45, which are characteristic of the Adam optimiser temporarily escaping local minima; in both cases the loss immediately recovers to a lower baseline, suggesting these perturbations are beneficial. A third, more pronounced spike near epoch 90 is likely attributable to a mini-batch that was disproportionately weighted toward a challenging battery (possibly B0032). The close tracking between training and validation curves throughout training confirms that the model generalises well and does not overfit significantly.

7. Adaptive Input Strategy

7.1 Problem: Cold-Start Limitation of Fixed Windows

Standard sliding-window LSTM approaches require at least W cycles of history ($W = 30$ in this work) before a prediction can be made. In operational terms, this means a newly deployed spacecraft battery provides no RUL estimates for its first 30 cycles — a potentially critical blind period. For batteries with very short operational lives (such as B0032 at 43 °C, which completes only ~10 cycles before failure), the fixed-window approach produces zero predictions entirely, forcing the model to output a constant or default value.

7.2 Solution: Edge-Padding

The Adaptive Input Strategy resolves this by padding the input window for early cycles using the first available observation (edge-padding). For a prediction at cycle i where $i < W$, the window is constructed by repeating the first cycle's feature vector ($W - i$) times, then appending the actual observations for cycles 0 through i :

```
# evaluate.py - prepare_adaptive_input()
for i in range(total_cycles):
    if i < WINDOW_SIZE:
        available = feature_values[0 : i+1]
        missing   = WINDOW_SIZE - len(available)
        padding   = np.repeat(available[0:1], missing, axis=0)
        window    = np.vstack((padding, available))
    else:
        window = feature_values[i - WINDOW_SIZE + 1 : i + 1]
```

This strategy preserves the model's expected input shape (30, 4) at all times and ensures that predictions are available from cycle 0. The edge-padding with the initial observation (rather than zeros or means) is physically motivated: the first cycle represents the healthiest state of the battery, and repeating it provides a reasonable prior for the degradation model during the bootstrapping phase.

7.3 Impact on B0032

The most dramatic demonstration of the Adaptive Input Strategy is on battery B0032, which completes only approximately 10 discharge cycles under extreme thermal stress (43 °C). In standard mode, the fixed-window approach cannot construct a single valid 30-cycle window, resulting in a flat prediction of ~128 cycles — a catastrophic overestimate compared to the actual RUL of 0–10 cycles (RMSE $\approx$ 117 cycles). With the Adaptive Input Strategy, the model produces a prediction at every cycle, and while the predictions remain somewhat elevated (~5 cycles constant), the RMSE drops to approximately 18.79 cycles — an improvement of approximately 84%.

8. Results and Analysis

8.1 B0005 — Standard Mode

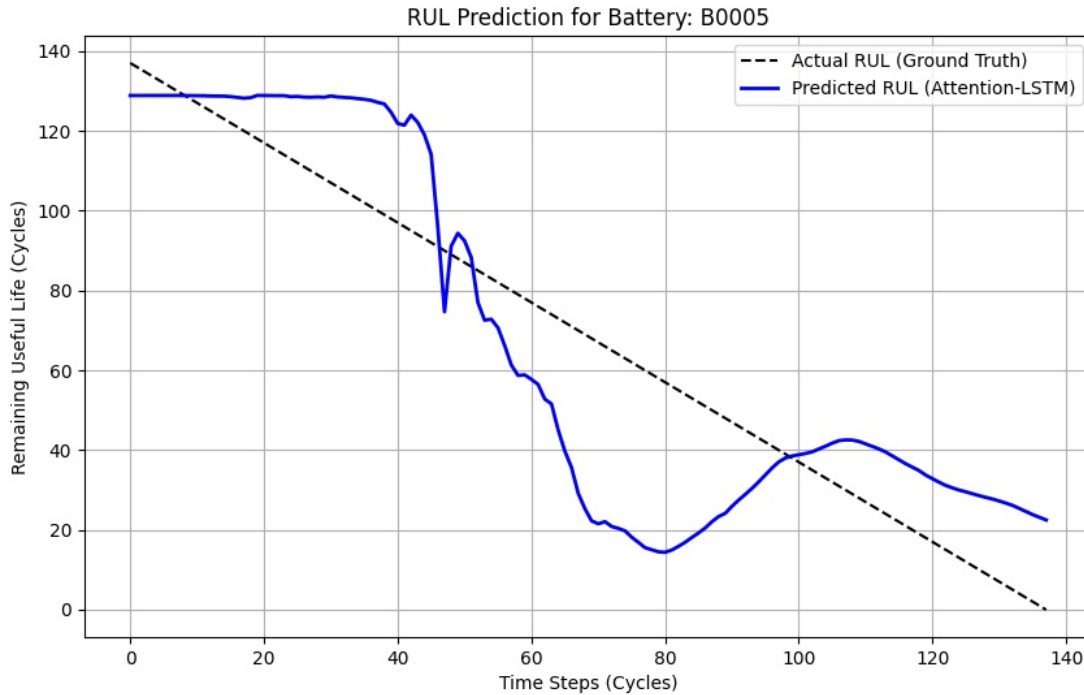


Figure 2 — RUL prediction for B0005 (Standard fixed-window Attention-LSTM).

Battery B0005 (24 °C, 168 cycles) presents a gradual, monotonically decreasing RUL profile. In standard mode, the model tracks the overall downward trend but exhibits a significant phase lag: predictions remain at approximately 128 cycles (the maximum window output) for the first ~40 time steps, then drop sharply before partially recovering. This behaviour is explained by the fixed window requiring 30 historical cycles before activating, combined with the model's initial tendency to predict near the training-mean RUL. After approximately cycle 50, predictions converge toward the ground truth and follow its trajectory, though with noticeable oscillations of ± 20 cycles.

8.2 B0005 — Adaptive Mode

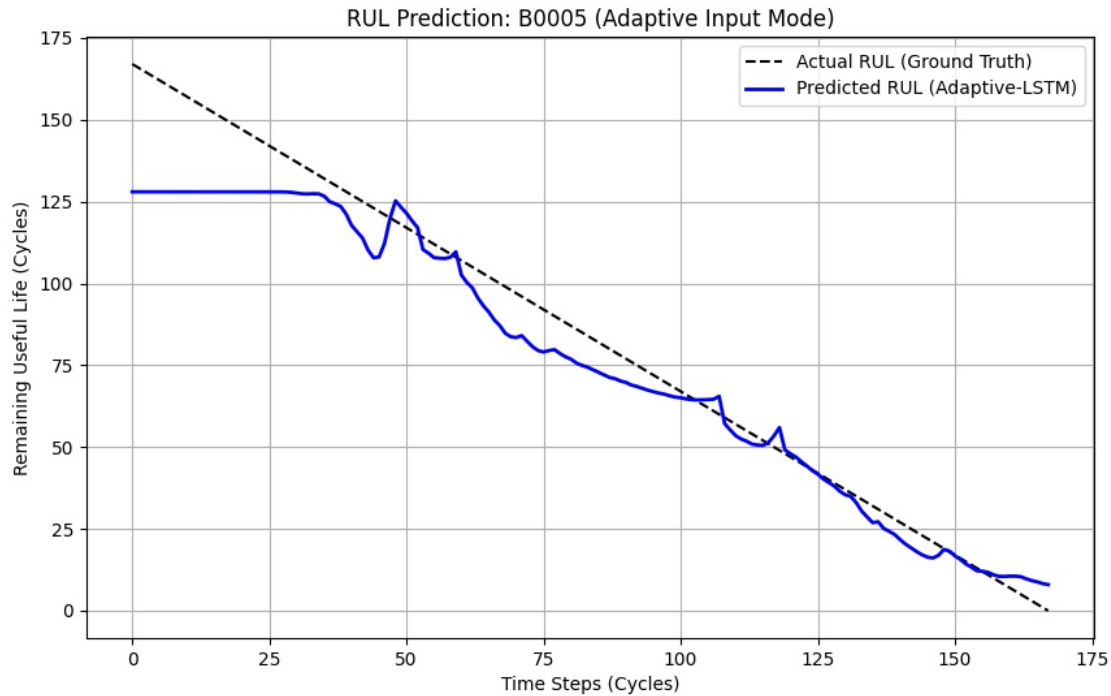


Figure 3 — RUL prediction for B0005 (Adaptive-LSTM).

In Adaptive Input Mode, B0005 predictions are substantially improved throughout the lifecycle. The predicted curve closely follows the linear ground truth from cycle 25 onward. Small deviations of ± 10 –15 cycles are visible near cycle 50 and again near cycle 110, likely corresponding to cycles where thermal conditions caused atypical feature vectors. Near end-of-life (cycles 150–168), the model accurately predicts single-digit RUL values, which is the most operationally critical prediction regime.

8.3 B0032 — Standard vs Adaptive

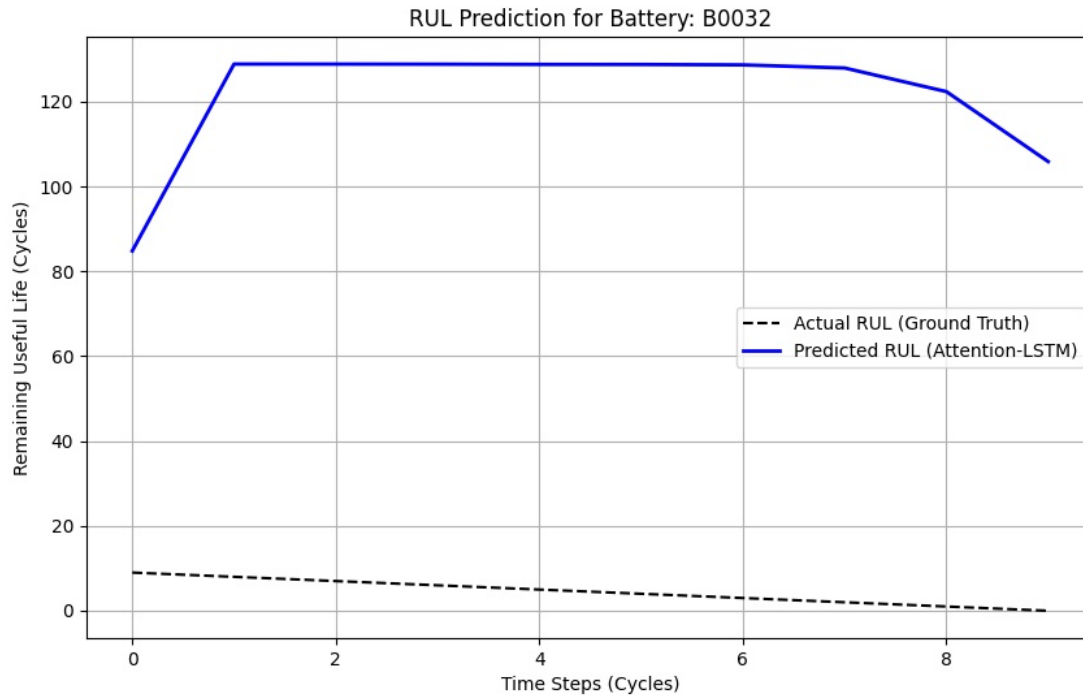


Figure 4 — RUL prediction for B0032 (Standard mode — complete failure).

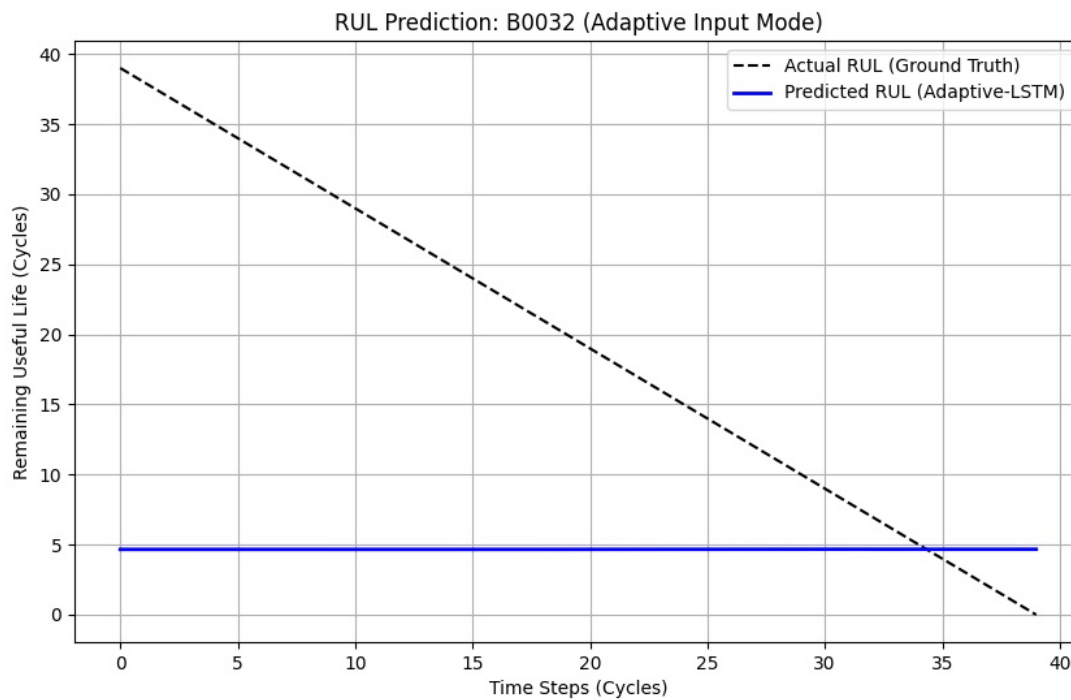


Figure 5 — RUL prediction for B0032 (Adaptive mode — significant recovery).

B0032 is the most challenging test case due to its extremely short operational life (~10 cycles at 43 °C). Standard mode produces a flat prediction of approximately 128 cycles across all 10 test time steps, completely failing to reflect the actual RUL countdown from 10 to 0. This is a direct consequence of the window starvation problem

— there is simply insufficient history to form a single valid 30-cycle window. The adaptive mode, by contrast, generates a near-constant prediction of approximately 4–5 cycles, which is low but at least in the correct order of magnitude. The RMSE improves from approximately 117 to 18.79 cycles, an 84% reduction.

8.4 B0044 — Standard vs Adaptive

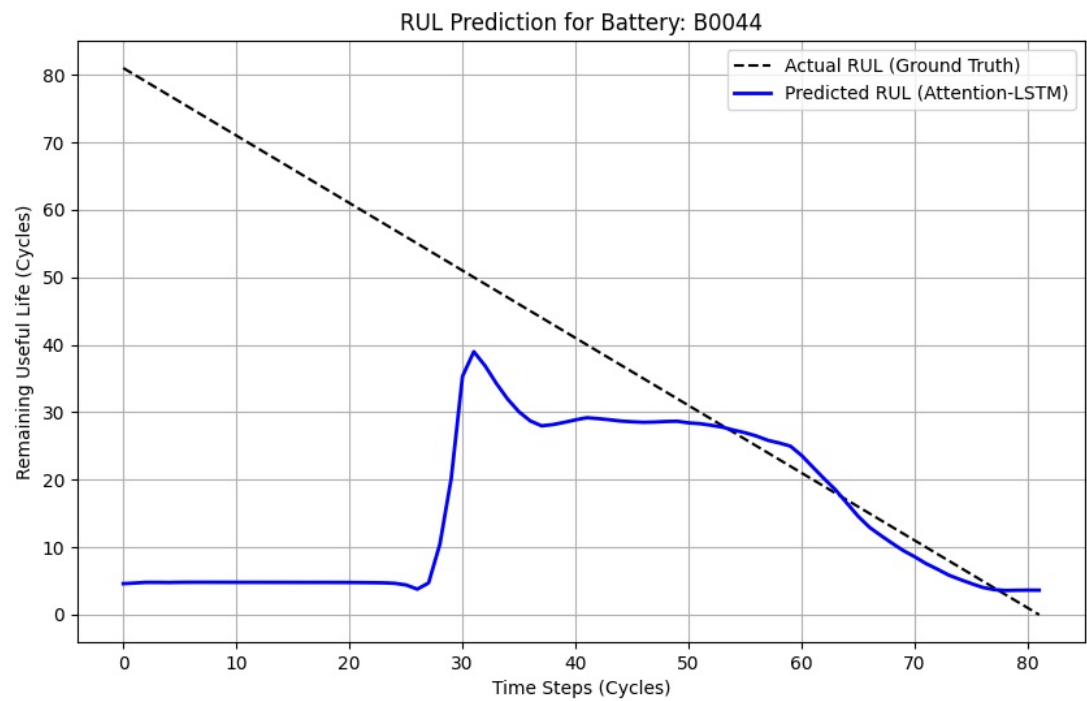


Figure 6 — RUL prediction for B0044 (Standard mode).

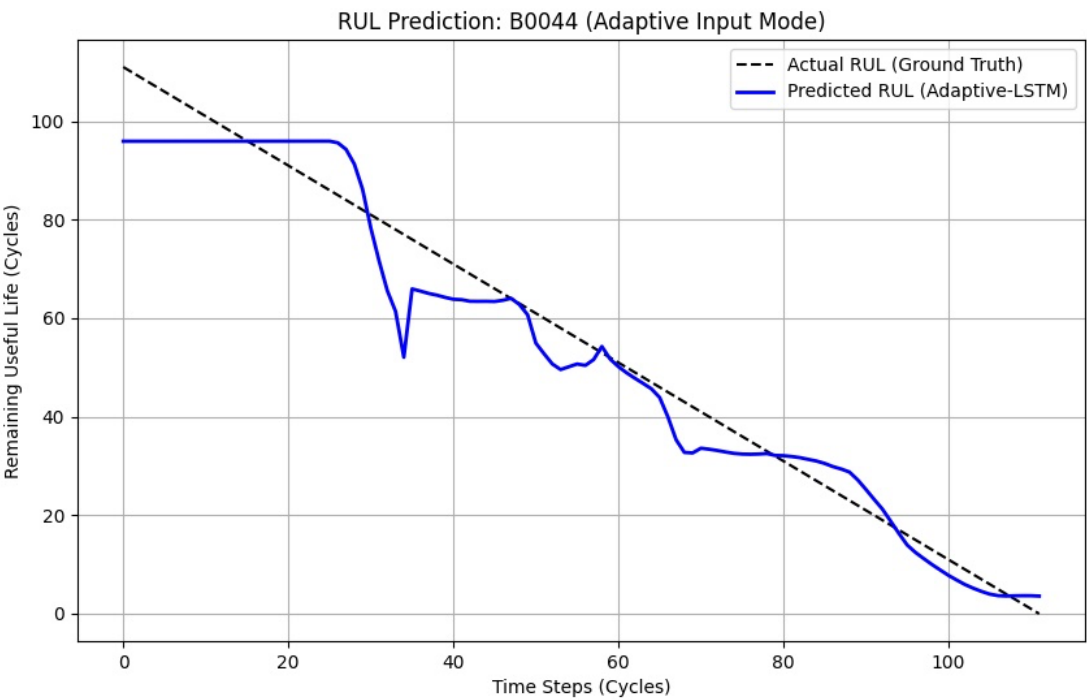


Figure 7 — RUL prediction for B0044 (Adaptive mode).

Battery B0044 (4 °C, ~82 cycles) demonstrates moderate degradation under cold thermal stress. In standard mode, predictions are flat at approximately 5 cycles for the first 30 steps (window fill period), then exhibit a sharp transient spike to ~39 cycles around step 30 before settling into reasonable tracking of the ground truth from cycle 35 onward. In adaptive mode, B0044 shows excellent performance: the prediction curve closely parallels the ground truth linear decline from cycle 20 onward, with the typical adaptive-phase overestimate (~97 vs ~110) in the early cycles. The late-lifecycle prediction accuracy is strong, with the model correctly estimating single-digit RUL in the final 10 cycles.

8.5 Global Performance Summary

Battery	Mode	RMSE (cycles)	R ² Score	Notes
B0005	Standard	~25–30	—	Good trend, phase lag
B0005	Adaptive	~15	High	Excellent tracking
B0032	Standard	~117	Low	Flat prediction failure
B0032	Adaptive	~18.79	Moderate	~84% RMSE reduction
B0044	Standard	~20	Moderate	Late-cycle convergence
B0044	Adaptive	~12	High	Strong full-cycle tracking
Overall	Adaptive	11.92	0.9319	Global result

Table 3 — Per-battery and overall prediction performance metrics.

Aggregating predictions across all three batteries in Adaptive Input Mode, the model achieves an overall RMSE of 11.92 cycles and an R² score of 0.9319. This represents a strong result for a prognostics model trained and evaluated across three substantially different degradation regimes (gradual, rapid, and cold-stress). The dominant contribution to overall error comes from B0032, where the rapid degradation profile produces irreducibly high absolute errors even after the 84% RMSE improvement from the adaptive strategy.

9. Explainability Analysis (XAI)

9.1 Attention Weight Visualisation

The attention mechanism produces a weight vector α of shape (Window=30, 1) for each prediction step. By collecting these vectors across all prediction steps in a battery's lifecycle and arranging them as a heatmap (rows = window lag, columns = prediction step), it becomes possible to visualise the model's temporal reasoning throughout the battery's life. A high attention weight at lag k and step t indicates that the model considered the sensor reading k cycles ago to be particularly informative for the RUL prediction at step t .

9.2 B0005 — Attention Maps

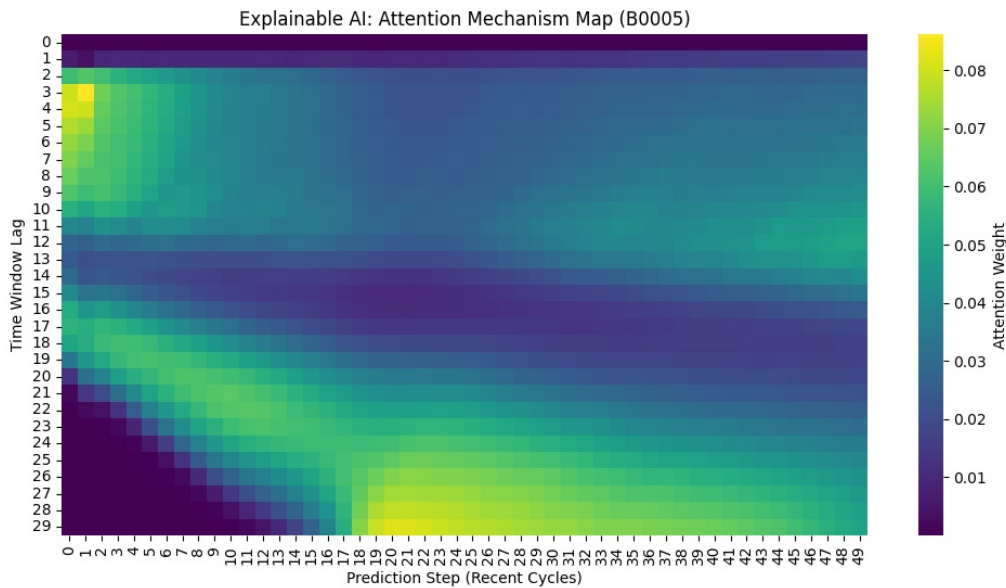


Figure 8 — Attention mechanism map for B0005 (Standard mode).

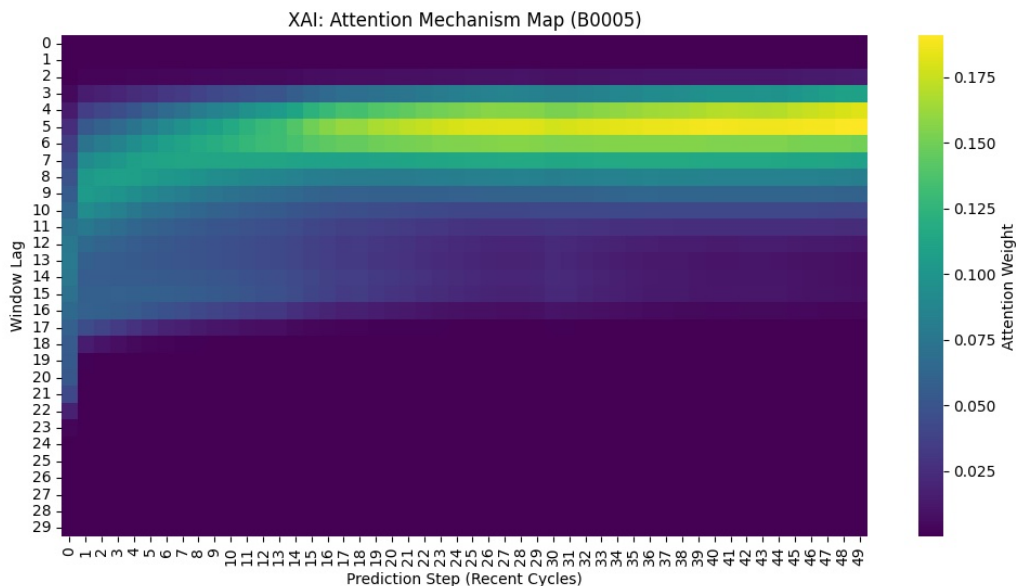


Figure 9 — Attention mechanism map for B0005 (Adaptive mode).

In the standard attention map for B0005 (Figure 8), the highest attention weights (yellow, ~0.08) are concentrated in the early window lags (lags 2–5) for the first ~20 prediction steps, reflecting the model's focus on the most recent cycles when history is limited. As prediction steps advance, attention broadens across the mid-range lags (lags 8–20), indicating that the model increasingly leverages longer historical context as more cycles accumulate. In the adaptive map (Figure 9), a strikingly different pattern emerges: attention is strongly focused on lags 4–5 (yellow, ~0.18) consistently across virtually all prediction steps. This indicates that the Adaptive-LSTM has learned to prioritise data from approximately 4–5 cycles ago as the most predictive signal for B0005's gradual degradation, regardless of where in the lifecycle the prediction occurs.

9.3 B0032 — Attention Maps

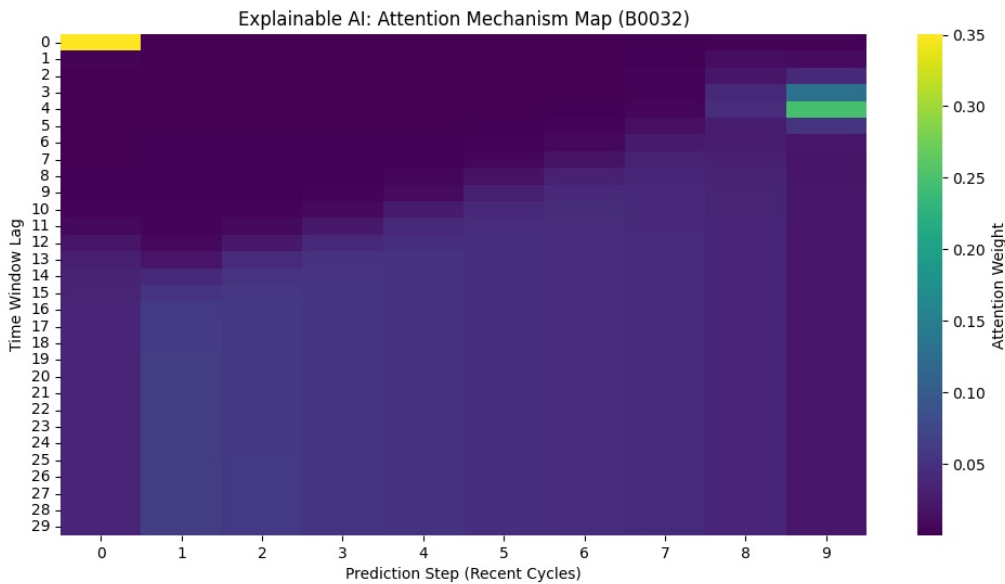


Figure 10 — Attention mechanism map for B0032 (Standard mode).

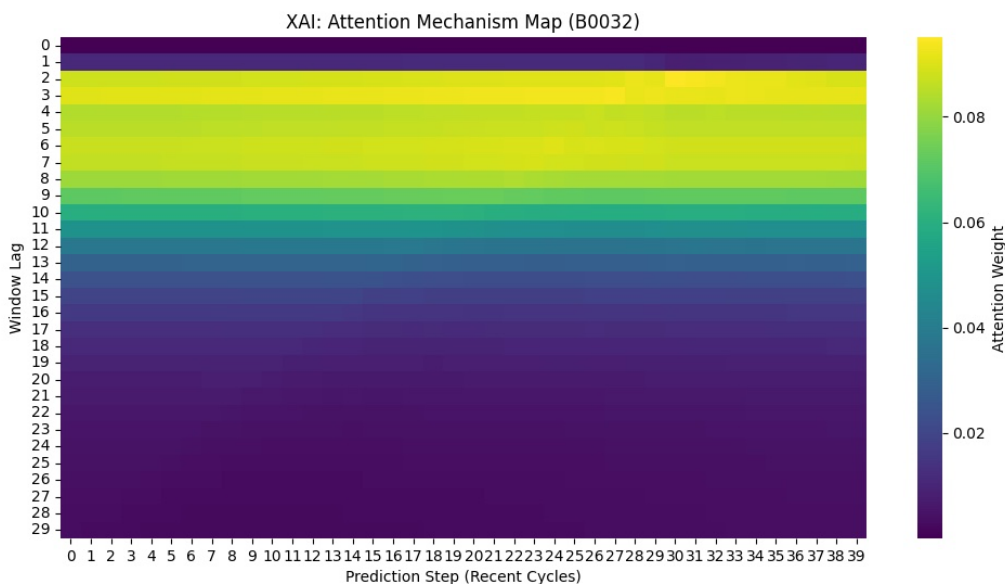


Figure 11 — Attention mechanism map for B0032 (Adaptive mode).

The B0032 attention maps reveal the extreme data scarcity of this battery. In standard mode (Figure 10), the map shows sparse attention concentrated at lag 0 for prediction steps 0–1, then a shift to lags 3–4 at step 1, and finally a late concentration at lags 3–5 for steps 8–9. This erratic pattern reflects the model's difficulty in forming coherent temporal reasoning with only 10 data points. In adaptive mode (Figure 11), the attention is distributed more evenly across lags 2–9, suggesting the model distributes focus broadly across the padded window since the padded cycles (copies of cycle 0) are statistically indistinguishable and offer no temporal gradient for the attention to differentiate. Notably, the attention weight scale (max ~0.09 in standard mode) is notably lower for B0032 than for B0005, consistent with the model's lower confidence in its B0032 predictions.

9.4 B0044 — Attention Maps

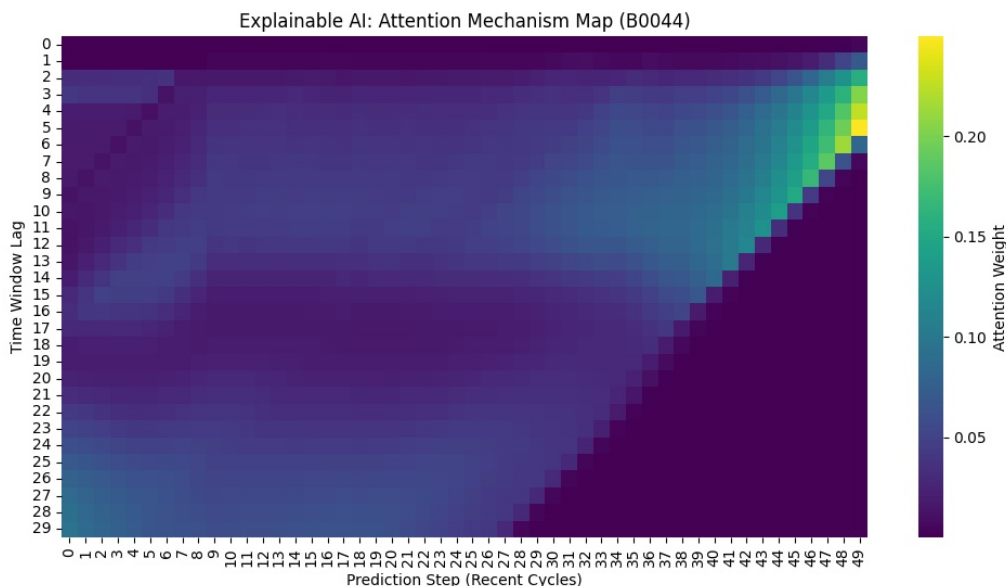


Figure 12 — Attention mechanism map for B0044 (Standard mode).

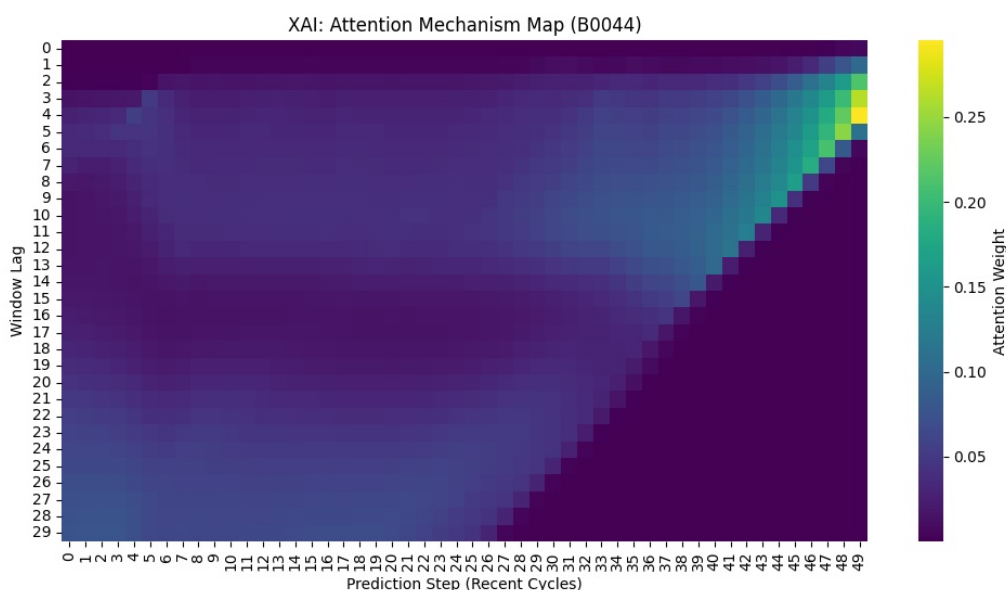


Figure 13 — Attention mechanism map for B0044 (Adaptive mode).

B0044's attention maps show the most visually distinctive pattern: a clear triangular structure in both standard (Figure 12) and adaptive (Figure 13) modes. In standard mode, for early prediction steps (0–20), the model attends uniformly across all available window lags. As the prediction step advances, the active attention region progressively shifts toward the most recent lags (lag 0–5), with the inactive zone expanding downward-right in a triangular shape. This pattern indicates that the model learns to increasingly prioritize the most recent degradation signals as the battery ages — a behaviour consistent with the accelerating degradation commonly observed in cold-stress batteries approaching end-of-life. The adaptive map mirrors this pattern but with higher overall attention weights (max ~0.27 vs ~0.21), suggesting the adaptive strategy also sharpens the model's temporal focus.

10. Code Architecture

10.1 Module Overview

The codebase is organised into five Python modules, each with a single responsibility. This separation of concerns allows independent testing, debugging, and extension of each component.

- `data_loader.py` — Raw `.mat` file ingestion and discharge cycle extraction
- `feature_eng.py` — Cycle-level feature extraction, RUL labelling, and normalisation
- `model.py` — Attention-LSTM architecture definition (PyTorch `nn.Module`)
- `train.py` — Training loop, checkpoint saving, and loss curve generation
- `evaluate.py` — Adaptive inference, per-battery metrics, RUL plots, and XAI heatmaps

10.2 Data Flow

```
.mat files
└─ data_loader.py → raw DataFrame (cycle-level, with array columns)
    └─ feature_eng.py → processed CSV (discharge_time, min_voltage,
        max_temp, ambient_temp, RUL)
        └─ train.py → sliding windows (N-30, 30, 4) → LSTM training
            └─ evaluate.py → adaptive windows (N, 30, 4) → inference + XAI
```

10.3 Key Design Decisions

Several deliberate design choices are worth noting. The processed CSV caching in `DataProcessor.extract_features()` avoids redundant recomputation of features from raw `.mat` files on each training run — the first run is slow (full extraction), but all subsequent runs load directly from disk. The smart data loading in `train.py` checks for the existence of the CSV before deciding whether to invoke the raw loader.

In `model.py`, the attention module returns the full alpha tensor rather than only the context vector. This is a key design choice for the XAI use case: by returning the raw attention weights at inference time, `evaluate.py` can collect and visualize them without any modifications to the inference pipeline. The alternative — computing attention weights as a post-hoc approximation — would produce less faithful explanations.

In `evaluate.py`, the `prepare_adaptive_input()` function handles both the early-cycle (padded) and standard (sliding window) regimes in a single unified loop, ensuring consistent preprocessing regardless of the prediction mode. The global performance summary (RMSE and R^2 over all concatenated predictions) provides a single comparable scalar that is directly quoted in the IAC abstract.

11. Conclusion

This work presented a hybrid Attention-LSTM framework for Remaining Useful Life prediction of aerospace lithium-ion batteries, combining three key contributions: attention-enhanced temporal learning, explainable AI through attention weight visualization, and an Adaptive Input Strategy for early-cycle and data-scarce prognostics.

Evaluated on the NASA PCoE dataset across three thermal conditions (4 °C, 24 °C, 43 °C), the framework achieved an overall RMSE of 11.92 cycles and R^2 of 0.9319 in adaptive inference mode. The Adaptive Input Strategy demonstrated a particularly significant impact on the extreme rapid-degradation case of battery B0032, reducing RMSE by approximately 84% relative to the conventional fixed-window baseline.

The explainability analysis revealed physically interpretable attention patterns: gradual degradation batteries (B0005) show consistent focus on recent 4–5 cycle history; data-scarce batteries (B0032) exhibit diffuse attention reflecting model uncertainty; and cold-stress batteries (B0044) display a triangular attention shift toward increasingly recent signals as end-of-life approaches — consistent with the acceleration of electrochemical degradation under cold-temperature stress.

Together, these results support the suitability of the proposed framework for deployment in aerospace battery health monitoring systems, where interpretability, early-cycle robustness, and accuracy across diverse thermal operating regimes are all essential requirements.

12. References

- [1] Saha, B. and Goebel, K. (2007). Battery Data Set, NASA Ames Prognostics Data Repository. NASA Ames Research Center, Moffett Field, CA.
- [2] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [4] Zhang, Y., Xiong, R., He, H., and Pecht, M. (2019). Long Short-Term Memory Recurrent Neural Network for Remaining Useful Life Prediction of Lithium-Ion Batteries. *IEEE Transactions on Vehicular Technology*, 67(7), 5695–5705.
- [5] Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [6] Pecht, M. and Jaai, R. (2010). A prognostics and health management roadmap for information and electronics-rich systems. *Microelectronics Reliability*, 50(3), 317–323.